

Group: 22BCP107 – Mann Patel & 22BCP118 – Rachit Shah

Instructions:

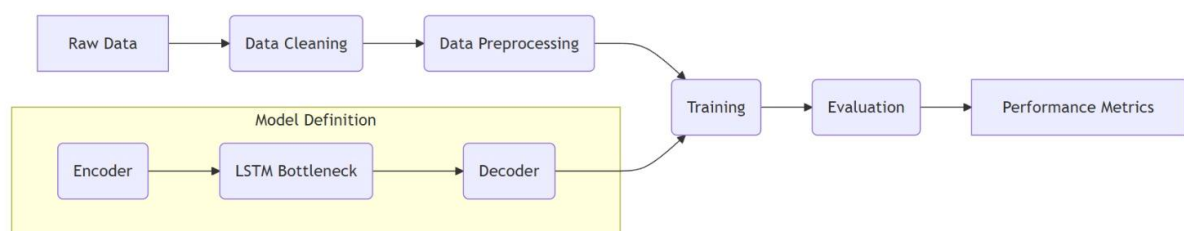
1. Design the Neural Network model for the project title submitted by you.
2. Demonstrate "Over-fitting" and solve the same using "Dropout technique".

Rubrics: (2 Marks each)

1. Demonstrate your project idea and implementation plan through a single diagram/mind map
2. Model Justification with respect to project domain
3. Demonstration of over fitting and dropout technique
4. Need of optimizer
5. Significance of your choice of optimizer

Title: Anomaly Detection in Credit Card Transactions

Project Idea and Implementation Plan



The system follows these steps:

- Data Collection: Raw credit card transaction data in CSV format
- Data Preprocessing: Normalization and handling of class imbalance
- Train-Test Split: Dividing data into training and testing sets
- Model Building: Neural network with input, hidden layers (including dropout), and output layer
- Training Phase: Using Adam Optimizer with a suitable learning rate
- Evaluation: Confusion Matrix and ROC Curve to assess performance
- Deployment: Prediction on unseen data

Model Justification with Respect to Project Domain

In the domain of **credit card fraud detection**, the primary challenge lies in identifying **anomalous patterns** in transactional data that deviate from normal user behaviour. These anomalies are rare and unpredictable, often requiring the model to **learn the structure of normal transactions** and flag deviations.

To address this, we chose an **LSTM-based Autoencoder**, which is well-suited to the problem for the following reasons:

1. Temporal Dependency Learning:

- LSTM (Long Short-Term Memory) networks are capable of capturing **sequential and temporal dependencies** in data.
- In credit card transactions, although the dataset may not appear strictly sequential, **user behaviour patterns over time** (e.g., time gaps, transaction frequency, amount trends) can still have implicit sequential correlations.
- LSTM helps the model learn these patterns more effectively than simple dense layers.

2. Unsupervised Learning Advantage:

- Fraud data is highly **imbalanced** (fraudulent cases are extremely rare).
- Autoencoders offer a natural **unsupervised approach**, where the model is trained only on **non-fraudulent (normal)** transactions to reconstruct them.
- During inference, if a transaction yields a **high reconstruction error**, it's flagged as anomalous (potential fraud).

3. Robustness via Encoder-Bottleneck-Decoder Structure:

- The encoder compresses high-dimensional input into a lower-dimensional latent space (bottleneck).
- The decoder attempts to reconstruct the input from this compressed representation.
- **Fraudulent data**, which doesn't fit the learned distribution, causes poor reconstruction, helping detect anomalies.

4. Dropout Regularization to Prevent Overfitting:

- To enhance generalization and prevent overfitting on the training set, **Dropout layers** are used in both encoder and decoder.
- This ensures the model does not memorize specific transaction patterns but learns generalized normal behaviour.

5. Scalability and Interpretability:

- LSTM Autoencoders are efficient for real-time inference once trained.
- Reconstruction error serves as a **clear anomaly score**, making detection explainable.

Demonstration of Overfitting and Dropout Technique

During training, overfitting is a major concern, especially with imbalanced datasets. To combat this, dropout layers were introduced within the neural network architecture. Dropout randomly disables a fraction of neurons during training, forcing the network to not rely on specific activations and improving generalization.

Overfitting Scenario (No Dropout):

- Without dropout, the model memorizes training data patterns.
- Training reconstruction loss drops while test loss increases.

Dropout Regularization:

- Dropout layers were introduced.
- Randomly deactivates neurons during training.
- Helps the model generalize and better detect unseen anomalies.

Need of Optimizer

In neural network training, an **optimizer** is essential for updating the model's weights in a way that minimizes the loss function. Without an optimizer, the model would not be able to learn from data or improve its predictions. The optimizer controls how the gradients (calculated via backpropagation) are applied to update the weights. The optimizer helps:

- Adjust weights dynamically to reduce errors.
- Speed up convergence by efficiently navigating the loss landscape.
- Improve model accuracy by refining weight updates in response to training data.

Significance of Your Choice of Optimizer

The model uses the **Adam Optimizer**, a widely used method that combines the benefits of AdaGrad and RMSProp. This is especially important in fraud detection, where many features may not contribute equally to classification. Adam also helps stabilize training in the presence of noisy gradients, which are common in imbalanced datasets.

Adam Optimizer:

- Combines adaptive learning rates and momentum.
- Well-suited for sparse, noisy, and imbalanced datasets.
- Requires minimal tuning and provides fast convergence.

Why Adam Was Chosen:

- Effective for deep autoencoders.
- Learns quickly and adjusts for sparse gradients.
- Provides stable results with fewer iterations.

Final Model Selection: LSTM Autoencoder With Dropout Layer and ADAM optimizer.